Aarhus University

Project number - 2026F25

Parallax - Menus and UI

Students:
Thea Teresa Lindgaard (202204350)
Niels Jakob Østerberg (202105879)
Kasper Stecher (202100451)
Kasper Bentsen (202103249)

Counsellor:
Lukas Esterle

Aarhus, May 29th 2026

Contents

Author: Niels Jakob Østerberg

1. Menu Design

This proof of concept focused on the design and implementation of a menu system for Unity. The menus were developed with an emphasis on usability, accessibility, readability, and scalability.

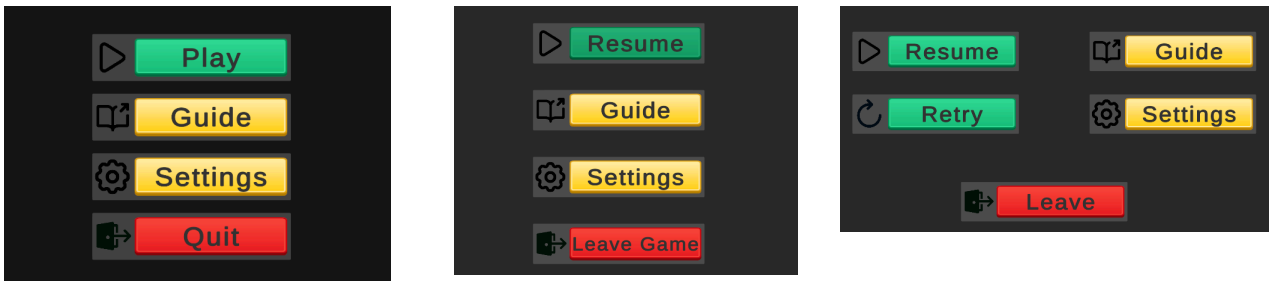
The menu system consists of three primary menus:

- MainMenu
- PlaybleLobby Menu
- Ingame Menu

The implementation includes navigation logic, visual feedback, settings management, guide pages, lobby creation, and multiplayer lobby interaction. The focus of this section is primarily on the UI implementation. Therefore, the underlying multiplayer and lobby networking systems are not discussed in depth.

The goal of the PoC was to establish a UI design that allows users to quickly identify interactions and navigate menus efficiently. In particular, Don Norman's design principles **[1]** and Gestalt principles **[2]** were used to support menu layout and interaction design. The overall objective was to create a menu system that feels intuitive, predictable, and easy to use.

Each menu contains a consistent visuals and navigation structure throughout the game. The lobby menu and in-game menu share a largely similar structure, where the in-game menu contains an additional retry option and a slightly modified layout to account for the extra interaction. The main menu focuses on navigation and access to the game's primary systems, see **Figure 1**.



MainMenu PlaybleLobby Menu Ingame Menu

Figure 1: Image of MainMenu, PlaybleLobby Menu, and Ingame Menu (Ingame menu as host)

1.1. Scanning Pattern and Layout Structure

Research presented in Text Scanning Patterns **[3]** explains that users commonly scan digital interfaces using an F-shaped scanning pattern. Users initially scan horizontally across the top of an interface before gradually moving downward with decreasing attention.

This behavior influenced the layout of the menu system. Frequently used actions were positioned near the top of menus, while less important or destructive actions were positioned lower. This reduces search time and effort while improving discoverability.

The implementation of this layout is visible in **Figure 1** above.

This layouts uses common game design conventions:

- **Play** and **Resume** being positioned at the top
- **Settings** and **Guide** being positioned in the middle
- **Quit** and **Leave Game** being positioned near the bottom

1.2. Don Norman's Design Principles

The following design principles from Don Norman **[2]** were considered during the implementation of the menu system.

1.2.1. Affordances

Buttons, sliders, toggles, and input fields were visually designed to communicate their functionality. Rounded button shapes, spacing, and contrasting colours help indicate that these elements are interactive.

1.2.2. Signifiers

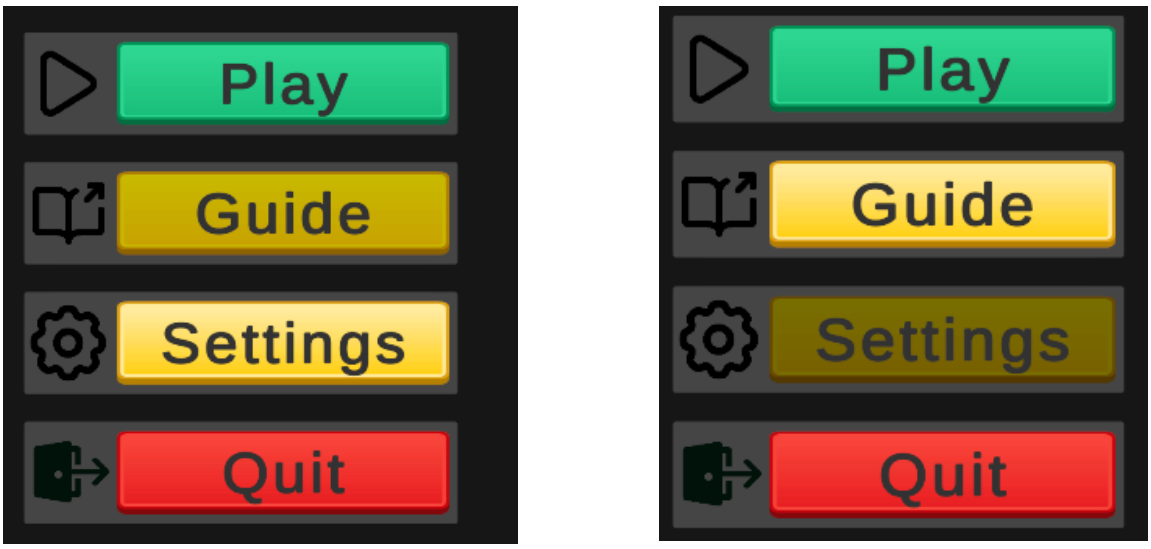
Visual indicators such as hover states, highlight colours, selected states, and icons were used to communicate interaction possibilities and current selection states.

1.2.3. Feedback

Immediate visual and audio feedback was implemented when navigating menus or selecting UI elements. This included:

- Button hover highlights
- Pressed button highlights
- Selection highlights
- Toggle and slider state changes

This feedback improves responsiveness and confirms user actions, see **Figure 5** of hover and selected highlights.



Selected highlight Hover highlight

Figure 5: Side by side of the Selected and Hover highlight

1.2.4. Constraints

Navigation paths were intentionally limited to prevent invalid interactions. Examples include disabled buttons, hidden navigation paths, and logical menu grouping.

1.2.5. Mapping and Consistency

The menus were designed using familiar video game conventions. Important actions are positioned near the top of menus, while destructive actions such as quitting are positioned lower.

The same visual language, colours, spacing, typography, and navigation structure were reused across all menus to improve consistency and reduce learning time.

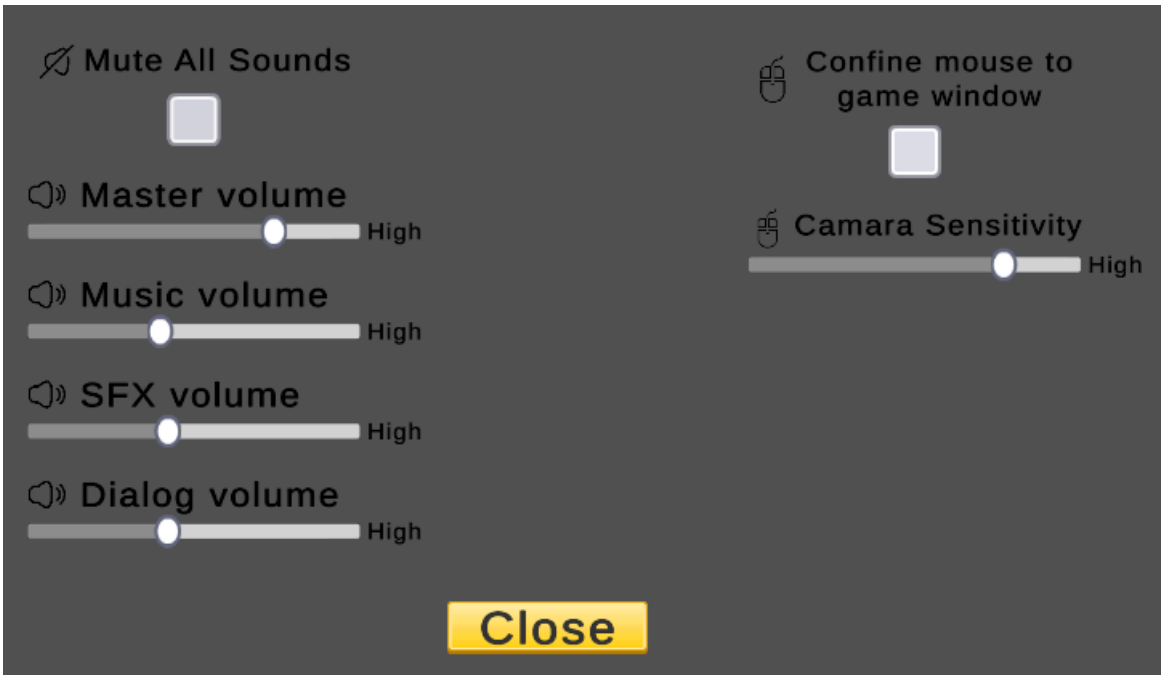


Figure 8: Image of the settings Menu

1.3. Gestalt Principles in Menu Design

Gestalt principles were used to improve visual hierarchy, readability, and menu organisation.

Principle	Application
Proximity	Related menu elements were grouped together to improve scanning efficiency and reduce visual clutter.
Similarity	UI elements with similar functionality use consistent colours, button styles, fonts, and interaction behaviour.
Hierarchy	Primary actions were emphasised using higher placement within the layout hierarchy.
Figure-Ground	Strong contrast between the dark background and brighter menu elements improves readability and accessibility.
Continuity	Menu navigation flow was designed to feel predictable and smooth. Navigation between menu pages follows consistent directional logic.

1.3.1. Examples using the Gestalt Principles

Proximity and **Similarity** were used in the settings and guide menus, see the settings menu in **Figure 8** where all the sound settings are on the left and the mouse related settings are on the right.

Figure-Ground was used in all the menus to improves readability, this can be seen in the bright colored buttons and text having a contrast to the background in all the menus.

Continuity was applied to menu navigation. Keyboard and controller navigation moves in the direction the user expects based on the input pressed.

1.4. General Menu Design Rationale

The menu system was designed as a navigation tool for Parallax. The layout focuses on:

- Clarity
- Readability
- Predictable navigation
- Fast interaction

The design allows for future expansion, by allowing additional pages and settings to be added without requiring very big restructuring. This aligns with Norman's principles of feedback and mapping, as well as Gestalt grouping principles.

1.4.1. UI Toolkit

Unity's newer UI Toolkit framework was also evaluated during development.

UI Toolkit uses:

- UXML for UI structure
- USS for styling

This workflow is heavily inspired by web development and functions similarly to HTML and CSS. UXML (Unity Extensible Markup Language) and USS (Unity Style Sheets).

UI Toolkit provides some long-term advantages:

- Better scalability for big products
- Reusable styling systems
- Cleaner separation of concerns (USS + UXML)
- More maintainable UI architecture
- Web-inspired workflows

The framework also supports debugging and testing workflows similar to web development. The UI Toolkit also introduces additional complexity:

- Higher learning curve
- Longer setup time
- Slower prototyping workflow
- Smaller ecosystem compared to Unity UI
- Less familiarity

For the project, rapid prototyping and implementation speed were prioritised.

The older Unity UI workflow together with TMP was therefore considered the most practical solution. Existing experience with the legacy Unity UI system also played an important role in this decision, as prior familiarity with Unity UI and TMP significantly reduced setup time and implementation overhead.

Choosing a familiar workflow allowed more development time to be allocated towards refining the settings system and implementing the PlayerPrefs functionality, rather than spending time learning and configuring a newer UI framework.

Although UI Toolkit offers a cleaner and more maintainable architecture for larger-scale projects, the increased setup complexity and development overhead would have reduced the time available for implementing and polishing core systems within the scope of the PoC.

1.4.2. Controller Support

The UI support both keyboard and controller input through Unity's Input System.

Menu navigation utilizes directional input, allowing players to navigate the interface without relying on a mouse cursor. This approach improves usability for controller players and creates a more consistent navigation flow across different input devices.

The controller navigation relies on Unity's built-in selectable UI navigation. Buttons and interactive UI elements are connected through navigation paths, ensuring predictable movement between interface elements when navigating vertically or horizontally.

Controller support also influenced several interaction design decisions, including:

- Large selectable UI elements
- Clear visual selection states
- Minimal navigation depth
- Consistent directional layouts

1.4.3. Navigation depth

The menu system was designed around a relatively shallow navigation structure to reduce unnecessary complexity and improve usability. Both the main menu and in-game menu follow a consistent hierarchical structure, where the user can access secondary systems such as settings, guides, and lobby management before returning to the previous state.

The diagrams in **Figure 9** and **Figure 10** illustrate the state flow and navigation depth of the implemented UI systems. The MainMenu UI acts as the primary entry point of the application and provides access to lobby creation, settings, guides, and application exit functionality. The Ingame UI follows a similar structure, allowing the player to pause gameplay and access settings, guides, retry functionality, or return to the main menu.

Special attention was given to minimising excessive navigation depth. Most user interactions can be completed within one or two transitions from the current menu state, which helps reduce friction and improves usability during gameplay. The guide system represents the deepest navigation layer, consisting of multiple sequential pages navigated through previous and next interactions.

The diagrams in **Figure 9** and **Figure 10** also demonstrate how individual menu states remain isolated and clearly separated, simplifying implementation and reducing the likelihood of unintended UI state conflicts during development.

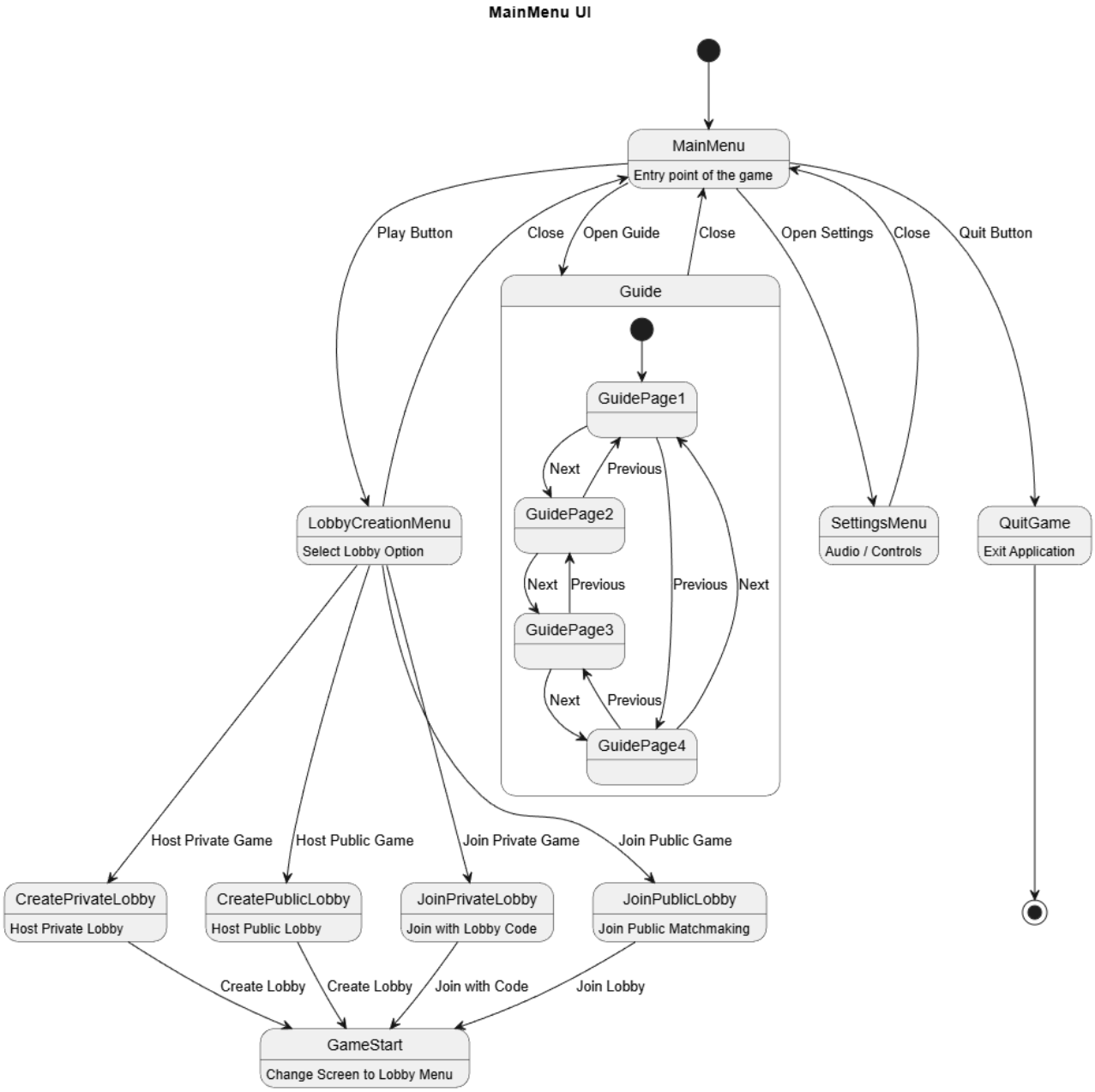


Figure 9: State diagram showing the navigation structure and menu flow of the MainMenu UI system.

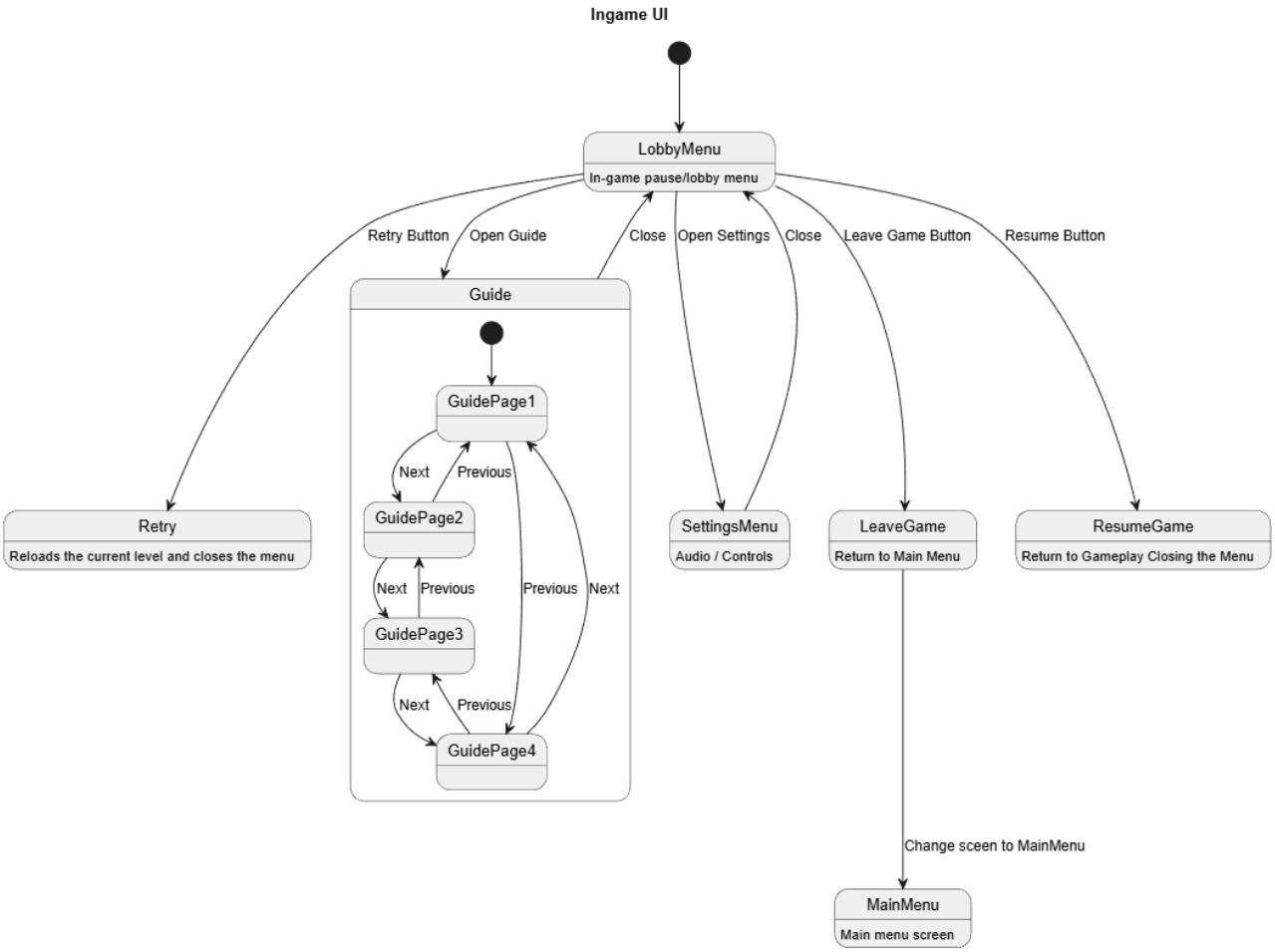


Figure 10: State diagram showing the navigation structure and menu flow of the in-game pause and lobby UI system.

1.4.4. Lobby Creation and Join System

The lobby creation interface shown in **Figure 11** was designed to support both public and private multiplayer sessions through a single unified workflow. The menu allows players to either create a new lobby or join an existing lobby depending on the selected option.

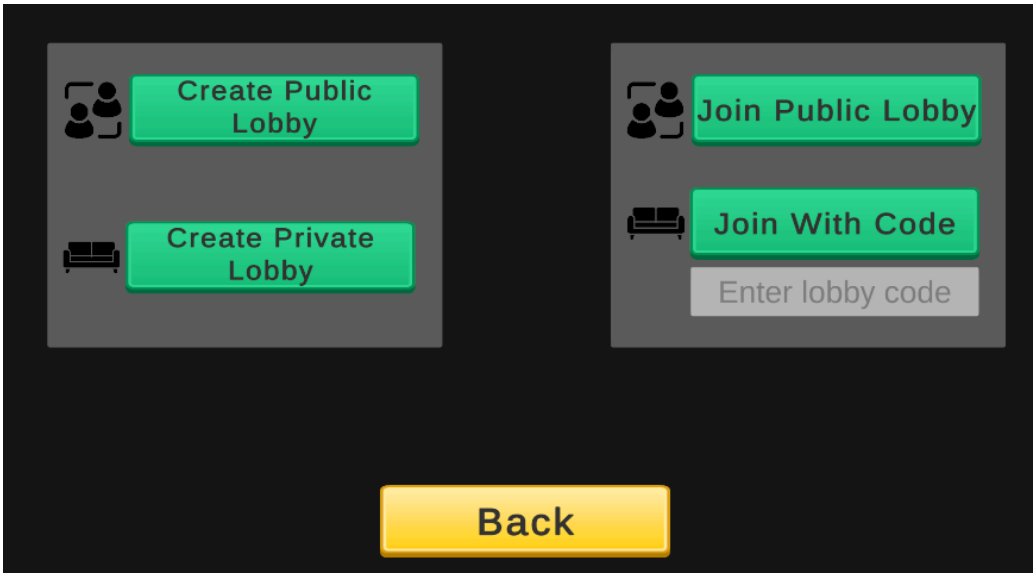


Figure 11: Lobby creation and joining interface

Two different lobby types were implemented:

- Public lobbies
- Private lobbies

Public lobbies allow any player to join through the `Join Public Lobby` button or by directly connecting using a generated lobby code see **Figure 12** to view the lobby code show in the top left corner. This provided flexibility, as players could either quickly join available sessions or connect to a specific public lobbys.

Private lobbies instead required a manually entered lobby code before access was granted. This restricted lobby access to invited players only and provided a more controlled multiplayer environment.

The interface was intentionally designed around a minimal navigation structure to reduce user friction during multiplayer setup. Large buttons, clear separation between hosting and joining actions, and direct code-entry functionality streamline the connection workflow while remaining easy to prototype.

1.5. Guide System

The guide was implemented to help players understand controls, gameplay mechanics, and the games systems. The guide consists of multiple pages that can be navigated using next and previous buttons. The guide uses large visuals and short text descriptions to improve readability.

The guide uses a script that handled page switching and ensures that only one guide page was active at a time. This simplified the navigation logic and made the system easy to expand with additional pages as they only need to be made and added to a list to work.

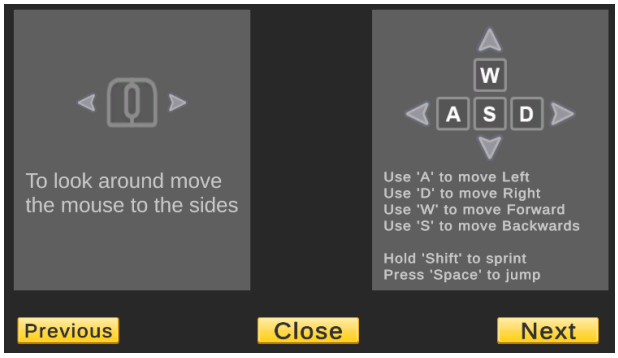
```
[SerializeField] private List<GameObject> pages;

private int currentPage;

void Start()
{...}

public void Next()
{
    pages[currentPage].SetActive(false);

    currentPage = (currentPage + 1) % pages.Count;

    pages[currentPage].SetActive(true);
}

public void Previous()
{
    pages[currentPage].SetActive(false);

    currentPage = (currentPage - 1 + pages.Count) % pages.Count;

    pages[currentPage].SetActive(true);
}
```

The `Next()` function increments the current page index and uses the modulus operator (`%`) to prevent overflow. When the index reaches the end of the list, the modulus operation wraps it back to `0`, effectively restarting from the first page.

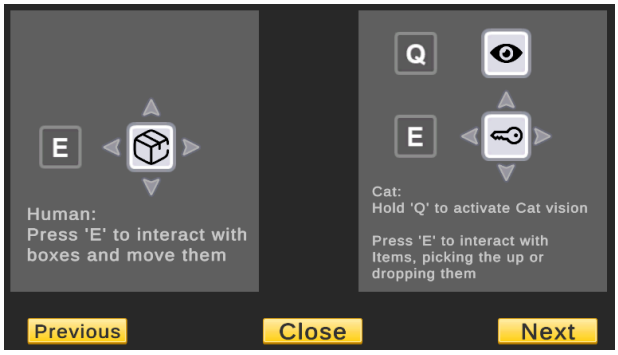
The `Previous()` function uses the same principle in reverse. Since decrementing could produce a negative value, `pages.Count` is added before applying the modulus operation. This ensures the index wraps correctly from `0` back to the last page, taking advantage of the list's zero-based indexing.

1.5.1. Guide Pages

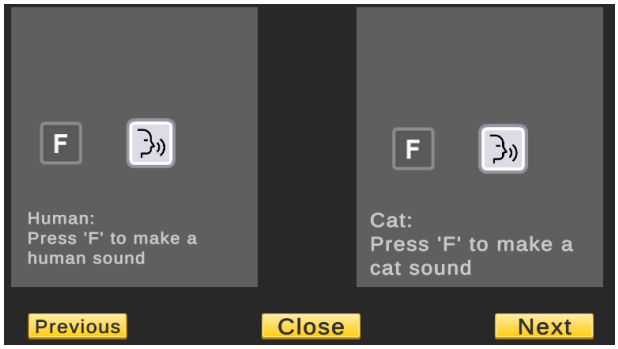
The guide system was divided into four pages, each focused on a specific aspect of gameplay. This separation reduced the amount of information shown at one time and letting players quickly navigate between topics using the next and previous buttons.



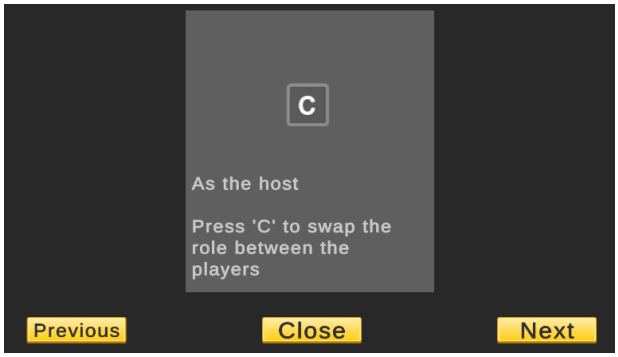
Page1: Player and Camera Movement



Page2: Game Object and World interactions



Page3: Human and Cat Dialog Sounds



Page4: Role Swap

Figure 13: Collection of all 4 guide pages

1.5.1.1. Page 1: Player and Camera Movement

The first guide page explains the basic movement controls used throughout the game. It introduces the keyboard controls for player movement, sprinting, and jumping, while also showing how mouse movement controls the camera direction. This page was designed to ensure that players could immediately understand how to navigate the environment before interacting with gameplay systems.

1.5.1.2. Page 2: Game Object and World Interactions

The second page focuses on interaction mechanics for both the human and cat characters. The human character can interact with and move boxes, while the cat character can interact with collectible items and activate cat vision. Presenting both interaction systems together, shows the differences between the two playable roles.

1.5.1.3. Page 3: Human and Cat Sounds

The third page explains the communication system used by both characters. Players can trigger different sound effects depending on whether they are controlling the human or the cat. This mechanic support player communication and reinforce the identity of each character through distinct audio feedback.

1.5.1.4. Page 4: Role Swap

The final page explains the role swapping mechanic available to the host player. By pressing the assigned key, the host can switch control between the human and cat roles. This page was included separately because the role swap mechanic directly affects cooperative gameplay and player coordination.

1.6. Settings System

As shown in **Figure 18**, the settings menu allows players to customise audio and gameplay-related settings. The interface was designed to provide quick access to commonly adjusted options while maintaining a simple and readable layout.

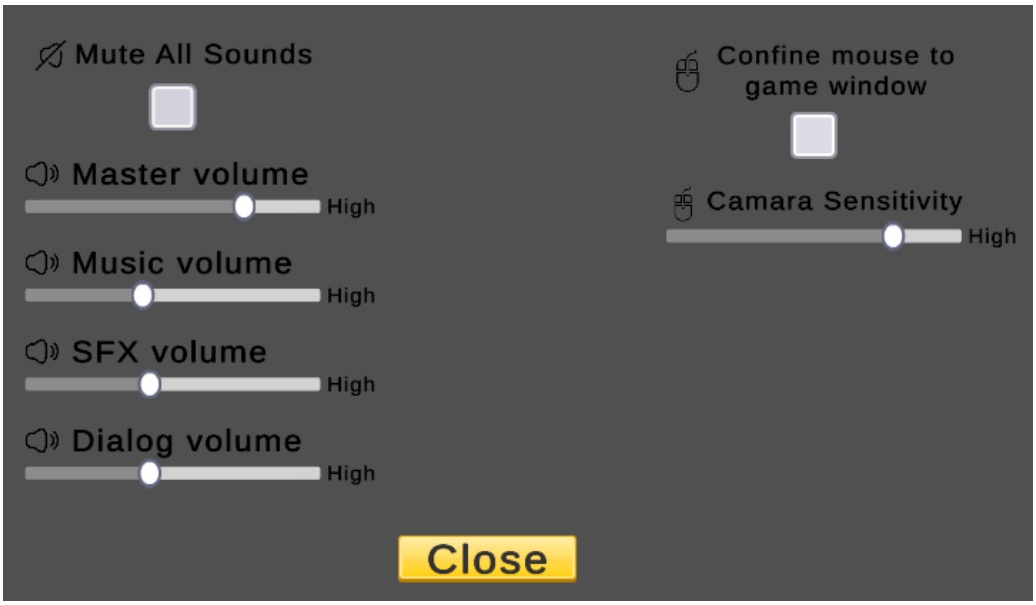


Figure 18: Settings menu interface

The menu includes several configurable options related to audio and gameplay behaviour:

- Master volume
- Music volume
- SFX volume
- Dialogue volume
- Camera sensitivity
- Toggle settings such as muting all sounds and confining the mouse to the game window

Audio settings were implemented using sliders, allowing players to adjust volume levels dynamically during gameplay. Toggle settings were implemented using checkboxes to provide clear visual feedback when enabled or disabled.

Settings values were stored using `PlayerPrefs`, allowing player preferences to persist between game sessions. The `SettingsManager` script handled saving and loading data, while a separate UI binder script `SettingsUIBinderV2` synchronised UI elements with the underlying settings values.

An event-driven architecture was used to propagate setting changes throughout the game. Unity's built-in `OnValueChanged` events on sliders and toggles were used to trigger setting updates whenever the player modified a value. These UI events passed updated values to the `SettingsManager`, which then broadcast the changes to subscribed systems.

1.7. Sprite Sheets

Sprite sheets were used throughout the menu system for icons and UI visuals. A sprite sheet stores multiple images within a single texture, allowing Unity to render UI elements more efficiently while reducing the number of draw calls.

Most UI sprites, including buttons and interface panels, were sourced from Kenney's UI pack **[4]**. Keyboard and controller input icons were sourced from Spriters Resource **[5]**, while additional gameplay and interaction icons were sourced from Iconmonstr **[6]**.

The sprite sheets were used for:

- Keyboard icons
- Navigation symbols
- Interaction prompts
- Button visuals
- Gameplay interaction icons

Using shared sprite sheets improved rendering efficiency by reducing texture switching and draw calls. It also improved visual consistency by ensuring that UI elements shared a unified visual style across the entire menu system.

The keyboard sprite sheets shown in **Figure 19** were particularly useful within the guide system, where gameplay controls and interaction prompts needed to be communicated clearly to the player.

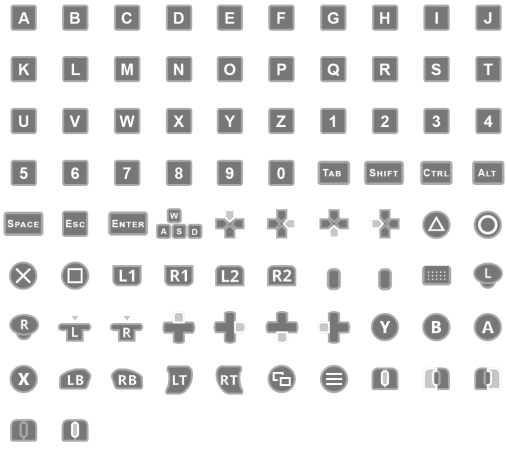


Figure 19: Controller Buttons & Keyboard Keys Icons sprite sheet

The Kenney UI pack shown in **Figure 20** provided the primary visual style for menu buttons and interface panels used throughout the project.

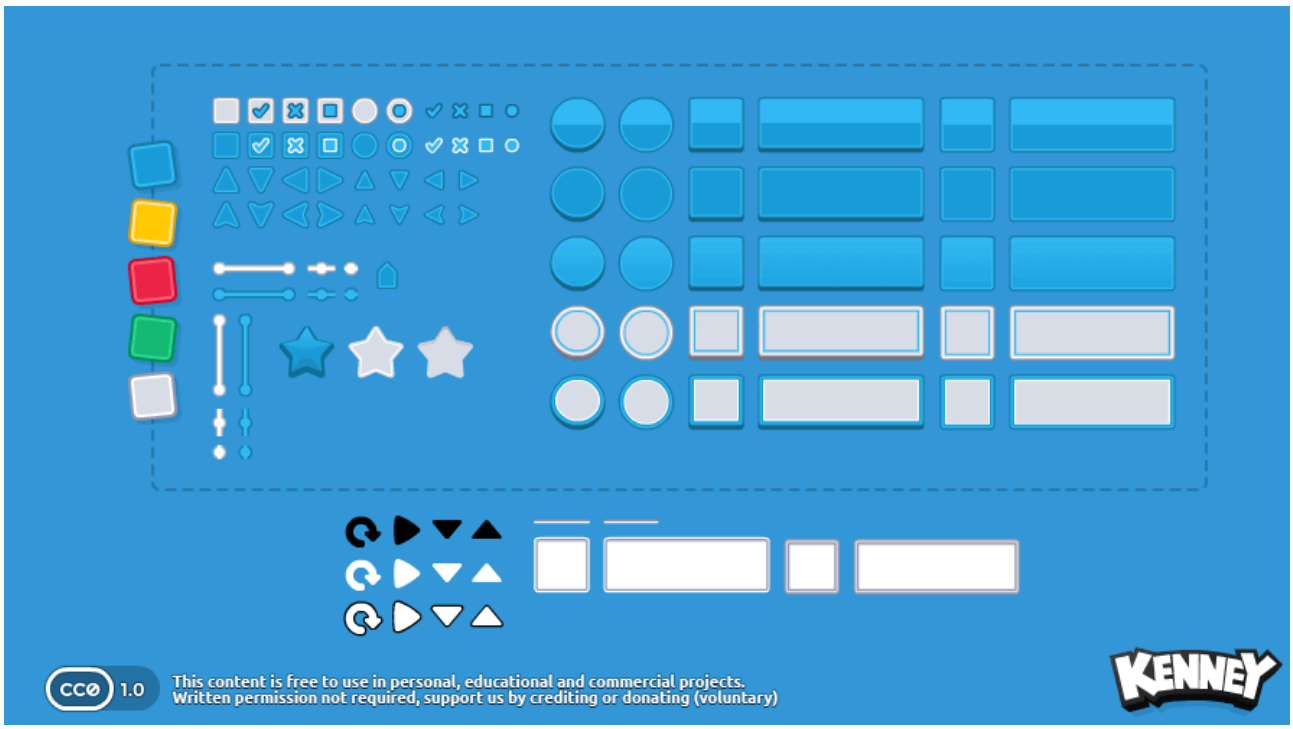
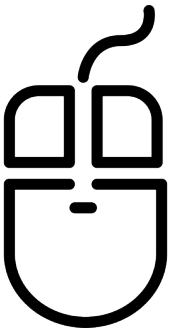
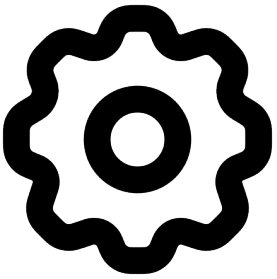


Figure 20: Examples of UI elements from the Kenney UI pack

Additional icons sourced from Iconmonstr were used throughout the menus to visually communicate button functions, settings categories, and gameplay interactions. These icons were also within the guide pages to provide visual explanations alongside text descriptions. Two examples are shown in **Figure 21**.



Mouse icon



Settings icon

Figure 21: Examples of icons sourced from Iconmonstr **[6]**

1.8. PlayerPrefs vs JSON

Evaluating two approaches for storing user settings data:

- JSON file storage
- PlayerPrefs

1.8.1. JSON File Storage

JSON storage provides a structured and flexible way to persist data. It is well suited for larger or more complex datasets where hierarchical organization is beneficial, such as:

- Save game systems
- Inventory data
- Character progression
- Nested configuration files (Structured data)

JSON also makes it easier to serialize and deserialize entire objects or collections of data. However, it introduces additional complexity, including file handling, serialization logic, and maintaining a structured schema. For very small datasets, this structure can become unnecessary overhead.

1.8.2. PlayerPrefs

PlayerPrefs is Unity's built-in lightweight persistence system. It stores simple key-value pairs such as:

- Sound settings
- Mouse sensitivity
- Toggle states
- Resolution preferences

PlayerPrefs was selected because the settings system only required small amounts of persistent data. Each setting element only needed to store a single primitive value, such as a boolean (true/false) or a numeric value. Since the total amount of data was minimal, the structured nature and flexibility of JSON did not provide significant advantages.

Using PlayerPrefs simplified the implementation by allowing each setting to be saved and retrieved independently through simple key-value access. This reduced both development complexity and maintenance overhead while remaining fully sufficient for the requirements of the settings menu.

Additionally, user tampering was not considered a significant concern, making PlayerPrefs an appropriate and efficient solution for this part of the game.

1.9. Event System and Setting Propagation

The settings system was implemented using an event-driven architecture. Instead of individual systems continuously checking for updated values, the `SettingsManager` propagated changes through events and subscriptions.

This approach reduced coupling between systems and simplified synchronization between UI elements and gameplay systems.

When a setting value was changed through the settings menu, Unity's built-in `OnValueChanged` callbacks on sliders and toggles triggered the corresponding setter methods within the `SettingsManager`. These methods updated the stored `PlayerPrefs` values and immediately invoked events associated with the modified setting.

- Audio systems subscribing to volume changes
- Camera systems subscribing to sensitivity changes
- UI systems subscribing to toggle-state updates

```
// Events
public event Action<float> OnMasterVolumeChanged;
public event Action<bool> OnMuteChanged;

// Key
private const string MasterVolumeKey = "master_volume";
private const string MuteKey = "mute_all";

// Setters
public void SetMasterVolume(Slider sliderValue) {
    PlayerPrefs.SetFloat(MasterVolumeKey, sliderValue.value);
    PlayerPrefs.Save();

    OnMasterVolumeChanged?.Invoke(GetMasterVolume());
}

public void ToggleMute() {
    bool muted = !GetMute();

    PlayerPrefs.SetInt(MuteKey, muted ? 1 : 0);
    PlayerPrefs.Save();

    OnMuteChanged?.Invoke(GetMute());
}

// Getters
public float GetMasterVolume() => PlayerPrefs.GetFloat(MasterVolumeKey, 1f);
public bool GetMute() => PlayerPrefs.GetInt(MuteKey, 0) == 1;
```

1.9.1. Publisher and Subscriber

The settings architecture used an event-driven publisher-subscriber pattern to propagate changes throughout the application.

- The `SettingsManager` functioned as the publisher
- Other systems acted as subscribers

When a player changed a setting through the UI, the value was passed to the `SettingsManager`, which stored the value using `PlayerPrefs` and then raised the corresponding change event. Subscriber systems listening for that event immediately updated their own state in response.

This design improved scalability because additional systems could respond to setting changes without modifying the existing settings logic.

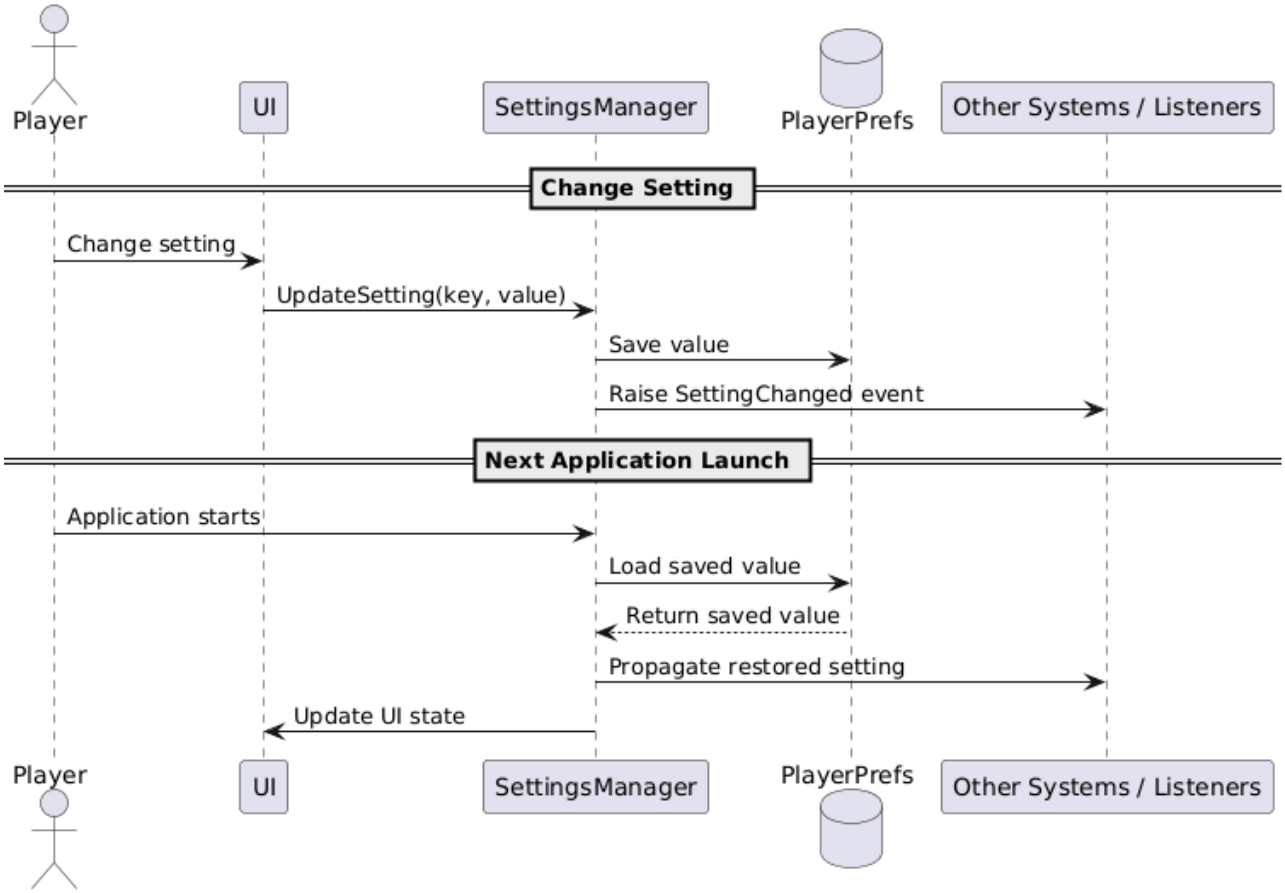


Figure 24: Sequence diagram showing how settings changes propagate through the application using the SettingsManager and PlayerPrefs system.

As illustrated in the sequence diagram in **Figure 24**, the architecture also restored persisted values during application startup. The `SettingsManager` loaded previously saved settings from `PlayerPrefs` and propagated the restored values back to both subscriber systems and UI elements. This ensured that user preferences remained persistent across application sessions.

Subscriber systems registered and removed event listeners during their lifecycle using `OnEnable()` and `OnDisable()`. This ensured that systems only responded to events while active and prevented invalid event references.

```
private void OnEnable() {
    var sm = SettingsManager.Instance;

    sm.OnMasterVolumeChanged += UpdateVolume;
    sm.OnMuteChanged += UpdateMute;
}

private void OnDisable() {
    var sm = SettingsManager.Instance;

    sm.OnMasterVolumeChanged -= UpdateVolume;
    sm.OnMuteChanged -= UpdateMute;
}

// Update methods
void UpdateVolume(float value) {
    masterVolumeSlider.SetValueWithoutNotify(value);
}

void UpdateMute(bool value) {
    muteToggle.SetIsOnWithoutNotify(value);
}
```

1.10. UI Framework and Text Rendering

The UI system was implemented using Unity's older UI workflow together with TextMesh Pro (TMP), rather than Unity's newer UI Toolkit framework.

TextMesh Pro was used for all text-based and interactive UI elements, including:

- Buttons
- Sliders
- Toggles
- Text
- Input fields

TMP provides improved text rendering quality compared to Unity's legacy text system. Features such as signed Distance Field (SDF) rendering, outline effects, dynamic scaling, and rich text formatting made it suitable for the menu system.

The following TMP features were especially useful:

- Sharp font rendering
- Better scaling support
- Rich text formatting
- Dynamic font sizing
- Improved readability

These features were important because the menu design relies heavily on strong visual contrast and readability across different resolutions and backgrounds.

Another major advantage of TMP is its support for SDF rendering. This allows text to remain sharp at different resolutions and scales without requiring multiple texture sizes, which is particularly important for games targeting multiple display resolutions.

1.11. Testing and Iterative Development

Testing of the menus and UI was performed continuously throughout the development process rather than being isolated to a final testing phase. Since the project followed an iterative prototyping workflow, UI functionality, navigation logic, settings propagation, and interaction feedback were repeatedly validated whenever new systems or gameplay features were implemented.

A large portion of the testing was conducted manually inside Unity during active development. This included verifying:

- Button navigation and selection behaviour
- Keyboard and controller input support
- Correct menu state transitions
- Visual feedback such as hover and selection highlights
- Persistence of settings through `PlayerPrefs`
- Proper propagation of settings events to subscribed systems
- Correct guide page navigation and wrapping behaviour
- Lobby creation and joining

Manual testing was especially useful when implementing gameplay systems that interacted with existing UI and audio functionality. For example, during development of the in-game radio, the audio settings system was used to ensure that:

- Master volume affected all audio sources correctly
- Music volume only affected music-related audio channels
- Mute functionality propagated properly across all active audio systems
- Audio changes updated immediately without requiring scene reloads
- Persisted settings restored correctly when returning to menus or reconnecting to lobbies

Testing of the settings system also occurred naturally during the development and testing of unrelated gameplay systems. While implementing and testing new features, the menu navigation and settings were frequently used as practical debugging and testing tools rather than as systems being directly tested themselves.

For example, during implementation of audio features such as the in-game radio with music, the audio settings were used to adjust and isolate specific sounds. Lowering dialogue and SFX volume made it easier to verify whether newly implemented audio sources and music systems were functioning correctly.

This resulted in the settings system and menu navigation receiving continuous indirect testing throughout development, since these systems were repeatedly interacted with during normal gameplay feature implementation and debugging workflows.

Bibliography

[1] I. D. Foundation, “ixdf - The Gestalt Principles.” [Online]. Available: https://ixdf.org/literature/topics/gestalt-principles

[2] D. Norman, “The Design of Everyday Things.” [Online]. Available: https://ia902800.us.archive.org/3/items/thedesignofeverydaythingsbydonnorman/The%20Design%20of%20Everyday%20Things%20by%20Don%20Norman.pdf

[3] K. Pernice, “Text Scanning Patterns: Eyetracking Evidence.” [Online]. Available: https://www.nngroup.com/articles/text-scanning-patterns-eyetracking/

[4] Kenney, “opengameart - UI Pack.” [Online]. Available: https://opengameart.org/content/ui-pack

[5] htauk, “spriters-resource - Controller Buttons & Keyboard Keys Icons.” [Online]. Available: https://www.spriters-resource.com/pc_computer/recroom/asset/178300/

[6] “iconmonstr.” [Online]. Available: https://iconmonstr.com/